Technical Documentation

AI-powered CRM follow-up email generation using LLM APIs

Version 1.0 | May 14, 2026

Prepared for: Engineering Team & Future Contributors

Table of Contents

1. **Project Overview**
2. **Architecture & System Design**
   - 2.1 How It Works
   - 2.2 Request / Response Flow
   - 2.3 Integration with the Parent CRM Service
3. **File Structure**
4. **Module Reference — email_generator.py**
   - 4.1 Data Models
   - 4.2 LLM Configuration
   - 4.3 Internal Functions
   - 4.4 API Endpoint
5. **LLM Provider Reference**
   - 5.1 OpenAI
   - 5.2 Groq
   - 5.3 Anthropic (Claude)
   - 5.4 Switching Providers
6. **Prompt Engineering**
   - 6.1 Prompt Structure
   - 6.2 Tone Map by Lead Temperature
   - 6.3 Output Format Contract
7. **API Reference**
   - 7.1 POST /email/generate-followup
8. **Configuration & Environment Variables**
9. **Setup & Integration Guide**
10. **Error Handling**
11. **Extension Guide**

1. Project Overview

The **Smart Email Generator** is a FastAPI router module designed to be plugged into an existing CRM + ML service. Given a lead's unique ID, it fetches the lead's full profile (including an AI-predicted temperature score of Hot / Warm / Cold) from the parent service, constructs a contextual prompt, calls a configured LLM, and returns a personalised follow-up email — subject and body — ready to send.

Key Features

- Plug-and-play FastAPI router — one import and one `include_router` call.
- Multi-provider LLM support: OpenAI, Groq, and Anthropic out of the box.
- Temperature-aware tone selection: Hot leads get urgent copy; Cold leads get re-engagement copy.
- Graceful no-key fallback: returns a placeholder email when `LLM_API_KEY` is not set, enabling development without credentials.
- Structured output parsing: always splits LLM output into discrete subject and body fields.
- Fully async: uses `httpx.AsyncClient` for non-blocking LLM calls.

Technology Stack

Component	Technology	Purpose
Web Framework	FastAPI	Router, request validation, async endpoint handling
HTTP Client	httpx	Async HTTP calls to LLM provider APIs
Data Validation	Pydantic BaseModel	Request/response schema enforcement
LLM — Default	OpenAI gpt-4o-mini	Primary email generation model
LLM — Alt 1	Groq (llama3-8b)	Low-latency alternative provider
LLM — Alt 2	Anthropic Claude	Claude-based generation (claude-3-haiku)
Config	os.getenv	Environment-based configuration, no config files needed
Logging	Python logging	Warning and error logging throughout
Language	Python 3.10+	Core runtime (uses tuple[str,str] type hint)

2. Architecture & System Design

2.1 How It Works

The module is a single FastAPI **APIRouter** mounted at the **/email** prefix. It depends on the parent application exposing an ML service via a `get_ml_service()` function in **main.py**. The email generator imports this function directly (same process, no HTTP overhead) to retrieve lead data with ML predictions already attached.

Step	Action	Detail
1	Client calls endpoint	POST /email/generate-followup with unique_id, deal_stage, past_communication
2	Fetch lead	_fetch_lead_from_service() calls ml_service.get_lead_with_prediction(unique_id)
3	Build prompt	_build_prompt() assembles lead details + tone guidance into an LLM prompt
4	Call LLM	_call_llm() dispatches to the configured provider (_call_openai / _call_groq / _call_anthropic)
5	Parse output	_parse_email_output() splits the LLM response into SUBJECT and BODY fields
6	Return response	EmailGenerateResponse is returned with success, lead info, subject, body, temperature

2.2 Request / Response Flow

```
Frontend / Client
    ■
    ■ POST /email/generate-followup
    ■ { unique_id, deal_stage, past_communication }
    ▼
email_generator.py → generate_followup_email()
    ■
    ■■ _fetch_lead_from_service(unique_id)
    ■     ■■ main.get_ml_service().get_lead_with_prediction(id)
    ■
    ■■ _build_prompt(lead, deal_stage, past_communication)
    ■
    ■■ _call_llm(prompt)
    ■     ■■ _call_openai()    (if LLM_PROVIDER=openai)
    ■     ■■ _call_groq()     (if LLM_PROVIDER=groq)
    ■     ■■ _call_anthropic() (if LLM_PROVIDER=anthropic)
    ■
    ■■ _parse_email_output(raw)
    ■
    ■■ return EmailGenerateResponse
         { success, lead_name, lead_email, subject, body, lead_temperature }
```

2.3 Integration with the Parent CRM Service

This module is **not a standalone service**. It is designed to be imported into a parent FastAPI application that already manages CRM leads with ML predictions. The only coupling point is the `get_ml_service()` function, which must be importable from `main` and return an object with a `get_lead_with_prediction(unique_id: str)` method.

Integration Requirement

- The parent app's main.py must expose: get_ml_service() → object
- That object must implement: get_lead_with_prediction(unique_id: str) → dict | None
- The returned lead dict must contain at least: name, email, ml_prediction.predicted_temperature
- Register the router: app.include_router(email_router) in main.py

3. File Structure

The project ships as a single module file intended to drop into an existing FastAPI project.

```
smart email generator/
    email_generator.py — Complete FastAPI router module
                         Contains: models, LLM clients, prompt builder,
                         output parser, and the /generate-followup endpoint

Parent project (expected, not included):
    main.py             — FastAPI app that imports and registers the router
    .env                 — Environment variables (LLM_API_KEY, LLM_PROVIDER, etc.)
```

4. Module Reference — email_generator.py

4.1 Data Models

EmailGenerateRequest

Pydantic model for the incoming POST request body.

Field	Type	Default	Description
unique_id	str	Required	The lead's unique ID in MongoDB
deal_stage	Optional[str]	"Prospect"	Current CRM deal stage (e.g. Prospect, Negotiation, Closed)
past_communication	Optional[str]	""	Free-text summary of previous interactions with this lead

EmailGenerateResponse

Pydantic model for the endpoint response.

Field	Type	Description
success	bool	Always True on a successful generation
lead_name	str	Full name of the lead from the CRM
lead_email	str	Email address of the lead
subject	str	Generated email subject line
body	str	Generated email body text
lead_temperature	str	AI-predicted temperature: Hot | Warm | Cold

4.2 LLM Configuration

Three module-level variables control which LLM is used. All are loaded from environment variables at import time, so changing them requires restarting the server.

Variable	Env Key	Default	Description
LLM_PROVIDER	LLM_PROVIDER	openai	Provider name: openai | groq | anthropic
LLM_API_KEY	LLM_API_KEY	(empty)	Secret key for the chosen provider
LLM_MODEL	LLM_MODEL	gpt-4o-mini	Model name; provider-specific

4.3 Internal Functions

All internal functions are prefixed with an underscore to signal they are not part of the public API. They should not be called directly from outside the module.

`_fetch_lead_from_service(unique_id: str) → dict` **[sync]**
Retrieves full lead data (including ML predictions) from the parent application's ML service. Imports `get_ml_service` from `main` at call time to avoid circular import issues. Raises HTTP 404 if the lead is not found, HTTP 500 on any other failure.

`_build_prompt(lead: dict, deal_stage: str, past_communication: str) → str` **[sync]**
Constructs the full LLM prompt string. Extracts name, role_position, skills, location, years_of_experience, and ml_prediction.predicted_temperature from the lead dict. Applies a tone instruction based on the lead temperature. Includes the past_communication summary or a default message if empty.

`_call_llm(prompt: str) → str` **[async]**
Top-level LLM dispatcher. If LLM_API_KEY is empty, returns a placeholder email string immediately (useful for local development). Otherwise delegates to the provider-specific function based on LLM_PROVIDER.

`_call_openai(prompt: str) → str` **[async]**
Calls the OpenAI Chat Completions API at api.openai.com. Uses temperature=0.7 and max_tokens=600. Also usable for any OpenAI-compatible endpoint by changing the base_url variable inside the function.

`_call_groq(prompt: str) → str` **[async]**
Calls the Groq API using the OpenAI-compatible chat completions format at api.groq.com. Defaults to llama3-8b-8192 if LLM_MODEL is not set.

`_call_anthropic(prompt: str) → str` **[async]**
Calls the Anthropic Messages API at api.anthropic.com using the 2023-06-01 API version header. Defaults to claude-3-haiku-20240307 if LLM_MODEL is not set.

`_parse_email_output(raw: str) → tuple[str, str]` **[sync]**
Parses the LLM's raw text response into (subject, body). Looks for a line starting with SUBJECT: for the subject and uses everything after BODY: as the body. Falls back gracefully: if SUBJECT is missing, uses 'Follow-up'; if BODY is missing, uses the entire raw string as the body.

4.4 API Endpoint

One POST endpoint is registered on the router:

```
@router.post("/generate-followup", response_model=EmailGenerateResponse)
async def generate_followup_email(request: EmailGenerateRequest)
```

Full path after mounting: **POST /email/generate-followup**. This is documented in detail in Section 7.

5. LLM Provider Reference

5.1 OpenAI (default)

Parameter	Value
Endpoint	https://api.openai.com/v1/chat/completions
Auth header	Authorization: Bearer {LLM_API_KEY}
Default model	gpt-4o-mini
temperature	0.7
max_tokens	600
Timeout	30 seconds

OpenAI-Compatible APIs

The _call_openai function uses a hardcoded base_url variable. To use any OpenAI-compatible API (Azure OpenAI, Together AI, Ollama, etc.), change base_url inside the function to the relevant endpoint.

5.2 Groq

Parameter	Value
Endpoint	https://api.groq.com/openai/v1/chat/completions
Auth header	Authorization: Bearer {LLM_API_KEY}
Default model	llama3-8b-8192 (if LLM_MODEL not set)
temperature	0.7
max_tokens	600
Timeout	30 seconds

5.3 Anthropic (Claude)

Parameter	Value
Endpoint	https://api.anthropic.com/v1/messages
Auth header	x-api-key: {LLM_API_KEY}
API version	anthropic-version: 2023-06-01
Default model	claude-3-haiku-20240307 (if LLM_MODEL not set)

Parameter	Value
max_tokens	600
Timeout	30 seconds

5.4 Switching Providers

Provider	LLM_PROVIDER	LLM_API_KEY source	Recommended LLM_MODEL
OpenAI	openai	platform.openai.com	gpt-4o-mini or gpt-4o
Groq	groq	console.groq.com	llama3-8b-8192 or llama3-70b-8192
Anthropic	anthropic	console.anthropic.com	claude-3-haiku-20240307 or claude-3-5-sonnet

6. Prompt Engineering

6.1 Prompt Structure

The prompt is a single user-turn message combining lead profile data, communication context, and precise output format instructions.

```
You are a professional B2B sales executive writing a follow-up email for a CRM lead.

LEAD DETAILS:
- Name: {name}
- Role/Position: {role_position}
- Skills: {skills}
- Location: {location}
- Years of Experience: {years_of_experience}
- Lead Temperature (AI Score): {predicted_temperature}
- Current Deal Stage: {deal_stage}

PAST COMMUNICATION SUMMARY:
{past_communication | 'No previous communication recorded.'}

INSTRUCTIONS:
Write a short, personalized follow-up email. Tone should be: {tone}.
The email must:
1. Address the lead by their first name
2. Reference their role/deal stage naturally
3. Include a clear call-to-action
4. Be 3-5 short paragraphs max

Return ONLY in this exact format:
SUBJECT:
BODY:
```

6.2 Tone Map by Lead Temperature

Temperature	Tone Instruction	Sales Strategy
Hot	urgent and enthusiastic — this lead is highly interested, push toward action	Minimise friction; drive to immediate next step (call, demo, contract)
Warm	friendly and informative — this lead is considering, nurture their interest	Provide value and social proof; maintain engagement without pressure
Cold	polite and value-focused — this lead needs re-engagement, remind them of benefits	Re-establish relevance; offer a low-commitment re-entry point

6.3 Output Format Contract

The prompt instructs the LLM to return output in a strict format. The parser (`_parse_email_output`) relies on this format:

- Line starting with `SUBJECT:` (case-insensitive) → extracted as the email subject.

- Line starting with `BODY:` (case-insensitive) → everything after this line is the body.
- If SUBJECT is absent → subject defaults to `"Follow-up"`.
- If BODY is absent → the entire raw response is used as the body.
- No extra text outside this format is expected; any preamble from the LLM will be captured in the body.

7. API Reference

7.1 POST /email/generate-followup

Property	Value
Full path	/email/generate-followup
Method	POST
Content-Type	application/json
Response model	EmailGenerateResponse
FastAPI tags	Email Generator
Auth	None (inherits parent app auth if configured)

Request Body

```
{
    "unique_id": "LEAD-ABC123",
    "deal_stage": "Negotiation",
    "past_communication": "Called last Tuesday. Interested in the Enterprise plan."
}
```

Success Response (200 OK)

```
{
    "success": true,
    "lead_name": "Priya Sharma",
    "lead_email": "priya@example.com",
    "subject": "Next Steps for Your Enterprise Plan, Priya",
    "body": "Hi Priya,\n\nThank you for our conversation last Tuesday...\n\nBest
regards,",
    "lead_temperature": "Hot"
}
```

Error Responses

HTTP Status	Condition	Detail message
404 Not Found	Lead unique_id does not exist in the ML service	"Lead '{unique_id}' not found"
500 Internal Server Error	ML service import or call failed	"Could not fetch lead details"
502 Bad Gateway	LLM API returned a non-200 status	"LLM API call failed" or provider-specific message
500 Internal Server Error	Unsupported LLM_PROVIDER value	"Unsupported LLM_PROVIDER: {value}"

8. Configuration & Environment Variables

Variable	Default	Required	Description
LLM_PROVIDER	openai	No	Which LLM provider to use: openai | groq | anthropic
LLM_API_KEY	(empty)	Yes*	API key for the chosen provider. *Module works without it (placeholder mode).
LLM_MODEL	gpt-4o-mini	No	Model name. Provider-specific defaults apply when not set.

No-Key Placeholder Mode

When LLM_API_KEY is empty or not set, the module returns a static placeholder email instead of calling the LLM. This allows the endpoint to be tested and integrated without a real API key. A WARNING is logged to alert developers that the key is missing.

Recommended **.env** file for production:

```
LLM_PROVIDER=openai
LLM_API_KEY=sk-...
LLM_MODEL=gpt-4o-mini
```

9. Setup & Integration Guide

Prerequisites

- An existing FastAPI application with a CRM + ML service (main.py exposing get_ml_service()).
- Python 3.10 or later.
- Packages installed: `fastapi httpx pydantic`.
- An API key for at least one supported LLM provider.

Integration Steps

Step 1: Copy the file

```
Place email_generator.py in the same directory as main.py.
```

Step 2: Install dependencies

```
pip install httpx
# fastapi and pydantic are already required by the parent app
```

Step 3: Register the router in main.py

```
from email_generator import router as email_router
app.include_router(email_router)
```

Step 4: Set environment variables

```
export LLM_PROVIDER=openai
export LLM_API_KEY=sk-your-key-here
export LLM_MODEL=gpt-4o-mini
```

Step 5: Start the server

```
uvicorn main:app --reload
```

Step 6: Test the endpoint

```
curl -X POST http://localhost:8000/email/generate-followup \
-H "Content-Type: application/json" \
-d '{"unique_id": "LEAD-001", "deal_stage": "Prospect"}'
```

Verifying the Integration

After starting the server, navigate to `http://localhost:8000/docs` to see the **Email Generator** tag and the **/email/generate-followup** endpoint in the FastAPI auto-generated Swagger UI. You can test it directly from the browser.

10. Error Handling

The module handles errors at three layers and always returns structured HTTP responses rather than propagating raw exceptions to the client.

Lead Fetch Errors

Scenario	Handling	HTTP Response
Lead not found	Raises HTTPException directly	404 with lead ID in detail
HTTPException from ML service	Re-raised as-is (preserves original status code)	Original status code
Any other exception	Logged at ERROR level; generic 500 raised to avoid leaking internals	500 with 'Could not fetch lead details'

LLM API Errors

Scenario	Handling	HTTP Response
API returns non-200	Error logged; HTTPException raised	502 Bad Gateway
No API key set	Warning logged; placeholder returned	200 with placeholder email
Unsupported LLM_PROVIDER	HTTPException raised immediately	500 with provider name in detail

Output Parsing Errors

The parser never raises. If the LLM returns unexpected output:

- Missing SUBJECT line → defaults to `"Follow-up"`.
- Missing BODY line → entire raw LLM response used as the body.
- Both missing → subject = "Follow-up", body = full raw response.

11. Extension Guide

Adding a New LLM Provider

- **1.** Add an async function `_call_newprovider(prompt: str) → str` following the same pattern as the existing provider functions.
- **2.** Add an `elif LLM_PROVIDER == "newprovider":` branch in `_call_llm()`.
- **3.** Update `LLM_PROVIDER` env var and set the appropriate `LLM_API_KEY`.

Customising the Prompt

All prompt logic lives in `_build_prompt()`. To personalise further:

- Add more lead fields from the `lead` dict (e.g. company_name, industry, last_purchase_date).
- Extend the `tone_map` dict with additional temperature categories if your ML model produces more granular scores.
- Add few-shot examples inside the prompt string to improve output consistency.
- Inject product-specific information (pricing tiers, features) to make emails more concrete.

Adding Email Delivery

The current module only generates email content; it does not send emails. To add delivery:

- Integrate **SendGrid**, **AWS SES**, or **SMTP** after the `_parse_email_output` step in the endpoint.
- Add a `send: bool = False` field to `EmailGenerateRequest` to make delivery opt-in.
- Log sent emails to MongoDB or the assignment_history collection for audit trails.

Streaming Responses

For large email bodies or slow LLMs, consider streaming the response to the client. FastAPI supports `StreamingResponse` with `async generators`. OpenAI and Anthropic both support streaming via `stream=True` / `stream=True` parameters in their respective APIs.

Caching Generated Emails

To avoid regenerating emails for the same lead and context on repeated calls, add a Redis or in-memory cache keyed by `hash(unique_id + deal_stage + past_communication)`. Cache TTL of 1–24 hours is recommended depending on how frequently lead data changes.

Decoupling from the Parent Service

Currently, `_fetch_lead_from_service()` imports from `main` directly. To make this module fully independent (e.g. for deployment as a microservice):

- Replace the direct import with an HTTP call to the parent service's `/lead/{unique_id}` endpoint.
- Add a `CRM_SERVICE_URL` environment variable pointing to the parent service base URL.
- Use `httpx.AsyncClient` (already a dependency) for the HTTP call.

End of Documentation | Smart Email Generator v1.0